

《计算机网络》实验报告

实验 2：设计可靠传输协议并编程实现

姓名：蒋衲言 学号：2313546

1 实验内容

本次实验需要利用数据报套接字在用户空间实现面向连接的可靠数据传输，功能包括：

1. **连接管理**：包括建立连接、关闭连接和异常处理。
2. **差错检测**：使用校验和进行差错检测。
3. **确认重传**：支持流水线方式，采用选择确认。
4. **流量控制**：发送窗口和接收窗口使用相同的固定大小窗口。
5. **拥塞控制**：实现 RENO 算法。

要求：

1. 实现单向数据传输，控制信息需要实现双向交互。
2. 给出详细的协议设计说明。
3. 给出详细的实现方法说明。
4. 利用 C 或 C++ 语言，使用基本的 Socket 函数进行程序编写，不允许使用 CSocket 等封装后的类。
5. 在规定的测试环境中，完成给定测试文件的传输，显示传输时间和平均吞吐率，并观察不同发送窗口和接收窗口大小对传输性能的影响，以及不同丢包率对传输性能的影响。
6. 编写的程序应该结构清晰，具有较好的可读性。

2 协议设计

2.1 简介

UDP 协议本身是不面向连接、且不保证可靠性的传输层协议。但 UDP 仅提供最基本的数
据报传输服务，并不限制应用层在其之上实现更复杂的通信机制。本实验基于 UDP 数据报套
接字，在应用层自行设计并实现**连接管理、差错检测、确认重传、流量控制与拥塞控制**机制，从
而实现一种“面向连接的可靠数据传输协议”，我将其称之为 RUDP (Reliable User Datagram
Protocol)。该实验本质上是对 TCP 协议核心思想的用户态复现，但一些细节部分也做了修改。

2.2 报文格式

我设计的报文格式如图1所示，下面对首部 Header 每个字段进行说明：

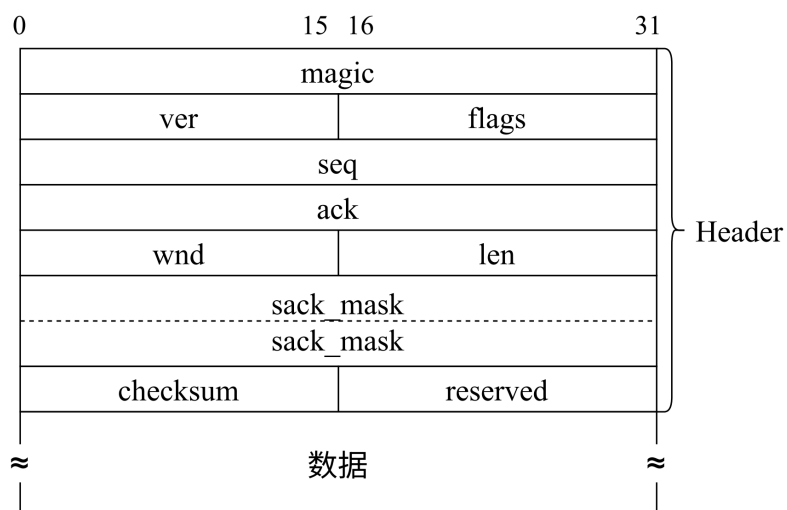


图 1: 报文格式

sender.cpp/receiver.cpp

```
#pragma pack(push, 1)
typedef struct Header {
    uint32_t magic;        // 协议标识
    uint16_t ver;          // 协议版本
    uint16_t flags;        // 控制标志位
    uint32_t seq;          // 序列号 (段号)
    uint32_t ack;          // 确认号 (累积确认)
    uint16_t wnd;          // 接收窗口大小
    uint16_t len;          // 数据长度
    uint64_t sack_mask;    // 选择确认SACK位图
    uint16_t checksum;     // 校验和
    uint16_t reserved;     // 保留字段
} Header;
#pragma pack(pop)
```

1. magic: 协议标识，这里是固定值 0x52554450，转换成 ASCII 字符就是 “RUDP”。

sender.cpp/receiver.cpp

```
static const uint32_t MAGIC = 0x52554450u;
```

2. ver: 协议版本号，当前固定为 1。

sender.cpp/receiver.cpp

```
static const uint16_t VER = 1;
```

3. **flags**: 控制标志位, 这是协议控制的核心字段, 多个标志可以按位组合, 如图2所示。有如下枚举变量:

sender.cpp/receiver.cpp

```
enum {  
    FLAG_SYN = 0x0001, // 建立连接  
    FLAG_ACK = 0x0002, // 确认  
    FLAG_FIN = 0x0004, // 关闭连接  
    FLAG_DATA = 0x0008, // 数据段  
    FLAG_RST = 0x0010 // 异常终止 (预留)  
};
```

SYN	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
ACK	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
FIN	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
DATA	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
RST	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
示例: SYN和ACK组合																
	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	

图 2: 控制标志位

4. **seq**: 序列号, 表示数据段号 (不是字节序号), 从 1 开始递增, 每个 DATA 包携带一个唯一的seq。
5. **ack**: 确认号 (累积确认), 指接收端 “下一个期望收到的段号”。
6. **wnd**: 接收窗口大小, 单位是段, 在实验中, 发送窗口 = 接收窗口 = 固定值W。
7. **len**: 数据长度, 表示 payload 的实际字节数, 满足 $0 \leq \text{len} \leq \text{MSS}$ 。
8. **sack_mask**: 选择确认位图, 这是协议中最重要的字段之一。bit $i = 1$ 表示序号为 $\text{ack} + i$ 的数据段已经被接收端收到 (但可能是乱序)。

例如, 假设接收端发送的包 $\text{ack} = 5$, $\text{sack_mask} = 0\text{b}11010$, 那么接收端在窗口内乱序收到了第 6 段、第 8 段、第 9 段, 但是第 5 段、第 7 段还没收到, 如图3所示。

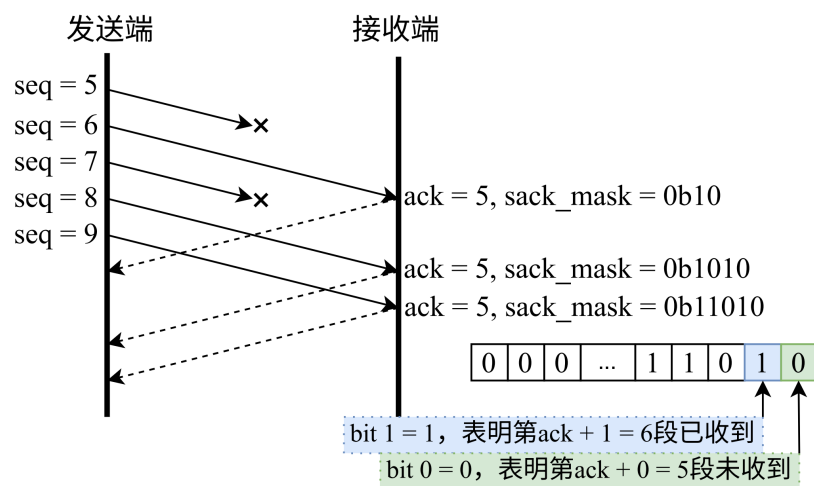


图 3: 选择确认位图示例

在实际的 TCP 协议中, SACK 的经典表示方式是“区间型 SACK”, 也就是通告每一个正确接收到的块的左边界和右边界。但是在本实验中, 明确规定发送窗口和接收窗口使用相同的固定大小窗口, 使用一个 W 位的 bitmap 是更简单、更直接的表达, 而未使用更加复杂的区间列表形式。

9. **checksum**: 校验和, 计算范围是整个 Header + payload, 发送前填 0, 计算后写入。接收端重新计算并比对。

10. **reserved**: 保留字段, 当前固定为 0。

除此之外, `#pragma pack(push, 1)` 的作用是强制 1 字节对齐 (禁止了编译器自动对齐), 防止编译器插入填充字节, 保证 Header 在内存中的布局和字节序严格连续。`#pragma pack(pop)` 恢复旧状态。

更多的协议设计具体细节见“程序编写”部分, 将和程序逻辑一同讲解。

3 程序编写

3.1 发送端 sender.cpp

3.1.1 校验和计算与构建数据包

sender.cpp

```
static uint16_t ones_complement_checksum(const uint8_t* data, size_t
len) {
    uint32_t sum = 0;
    size_t i = 0;
    while (i + 1 < len) {
        uint16_t word = (uint16_t(data[i]) << 8) | uint16_t(data[i +
1]);
        sum += word;
    }
}
```

```

        if (sum & 0x10000) sum = (sum & 0xFFFF) + 1;
        i += 2;
    }
    if (i < len) {
        uint16_t word = uint16_t(data[i]) << 8;
        sum += word;
        if (sum & 0x10000) sum = (sum & 0xFFFF) + 1;
    }
    return (uint16_t)(~sum & 0xFFFF);
}
// ...
static int build_packet(uint8_t* outbuf, int outcap,
    uint16_t flags, uint32_t seq, uint32_t ack,
    uint16_t wnd, uint64_t sack_mask,
    const uint8_t* payload, uint16_t len) {
    if (outcap < (int)sizeof(Header) + (int)len) return -1;

    Header h;
    h.magic = htonl(MAGIC);
    h.ver = htons(VER);
    h.flags = htons(flags);
    h.seq = htonl(seq);
    h.ack = htonl(ack);
    h.wnd = htons(wnd);
    h.len = htons(len);
    h.sack_mask = htonll_u64(sack_mask);
    h.checksum = 0;
    h.reserved = 0;

    memcpy(outbuf, &h, sizeof(Header));
    if (len > 0 && payload) {
        memcpy(outbuf + sizeof(Header), payload, len);
    }

    uint16_t cksum = ones_complement_checksum(outbuf, sizeof(Header) +
        len);
    uint16_t cksum_n = htons(cksum);
    memcpy(outbuf + offsetof(Header, checksum), &cksum_n, sizeof(
        uint16_t));

    return (int)(sizeof(Header) + len);
}

```

build_packet()函数用于构建网络数据包：首先检查输出缓冲大小 (outcap)；接着准备

要发送的 Header（本地临时变量h）并进行网络字节序转换；然后将 Header 和 payload 依次拷贝到输出缓冲outbuf；最后调用函数ones_complement_checksum()，计算校验和并写回报文的checksum字段位置。

ones_complement_checksum()函数用uint32_t sum做累加器，因为计算过程中可能会产生进位，16 位整型可能会溢出。遍历输入字节数组，按两个字节（即 word）为一组构成 16 位大端字（(data[i] << 8) | data[i+1]）并加到sum上。每次加后检查sum & 0x10000，若有溢出位则把高位折回（即把进位加回低 16 位）：sum = (sum & 0xFFFF) + 1。若总字节数为奇数，则最后一个字节被当作高字节，低字节补0，代码用uint16_t word = uint16_t(data[i]) << 8; sum += word; 实现。

3.1.2 数据包的解析

sender.cpp

```
static int parse_packet(const uint8_t* buf, int n, Header* out_h,
    const uint8_t** out_payload, uint16_t* out_len) {
    if (n < (int)sizeof(Header)) return 0;

    Header h;
    memcpy(&h, buf, sizeof(Header));

    uint32_t magic = ntohl(h.magic);
    uint16_t ver = ntohs(h.ver);
    if (magic != MAGIC || ver != VER) return 0;

    uint16_t recv_ck = ntohs(h.checksum);

    // 校验时把checksum字段置0再算
    uint8_t* tmp = (uint8_t*)malloc(n);
    if (!tmp) return 0;
    memcpy(tmp, buf, n);
    uint16_t zero = 0;
    memcpy(tmp + offsetof(Header, checksum), &zero, sizeof(uint16_t));
    uint16_t calc = ones_complement_checksum(tmp, n);
    free(tmp);

    if (calc != recv_ck) return 0;

    uint16_t len = ntohs(h.len);
    if ((int)sizeof(Header) + (int)len > n) return 0;

    *out_h = h;
    *out_payload = buf + sizeof(Header);
    *out_len = len;
```

```

    return 1;
}

```

`parse_packet()`函数用`memcpy(&h, buf, sizeof(Header))`读取报头（字段仍是网络字节序），然后用`ntohl(h.magic)`和`ntohs(h.ver)`检查`magic==MAGIC`且`ver==VER`，否则返回失败。接着提取报文中声明的校验和，计算并检验。最后再从报头读取`len = ntohs(h.len)`，如果`sizeof(Header) + len > n`则返回失败（声明的payload长度超出实际包长）。若所有检验都通过，则设置`out_h`、`out_payload`和`out_len`。

3.1.3 发送函数封装

sender.cpp

```

static int should_drop(double lossP) {
    if (lossP <= 0.0) return 0;
    double x = (double)rand() / (double)RAND_MAX;
    return (x < lossP) ? 1 : 0;
}
// ...
uint8_t sbuf[2048];
auto send_pkt = [&](uint16_t flags, uint32_t seq, uint32_t ack,
    uint64_t sack_mask, const uint8_t* payload, uint16_t len) -> int {
    int plen = build_packet(sbuf, (int)sizeof(sbuf), flags, seq, ack,
        (uint16_t)W, sack_mask, payload, len);
    if (plen < 0) return 0;
    if (should_drop(lossP)) {
        return 1;
    }
    int r = sendto(sock, (const char*)sbuf, plen, 0, (sockaddr*)&peer,
        sizeof(peer));
    return (r == plen) ? 1 : 0;
};

```

这里使用了一个`send_pkt`变量接收一个匿名函数（Lambda 表达式），用来封装发送函数。调用`build_packet()`函数构造报文，然后调用`should_drop()`函数进行丢包模拟；若包没有被丢掉，则用`sendto()`函数发送。

`should_drop()`函数传入丢包概率`lossP`作为参数，生成一个伪随机浮点数`x = rand() / RAND_MAX`（近似均匀分布在 $[0, 1]$ ），然后判断`x < lossP`：若真则返回 1（丢包），否则返回 0。

3.1.4 连接管理：三次握手

sender.cpp

```
uint64_t hs_deadline = 0; // 握手重传截止时间
int hs_retries = 0; // 握手重传次数

// SYN: flags=SYN, seq=0, ack=0
hs_deadline = now_ms(); // 立即发送
int connected = 0; // 连接成功标志

while (!connected) {
    uint64_t now = now_ms();

    // 超时重传SYN (包括第一次发送)
    if (now >= hs_deadline) {
        if (hs_retries >= 10) {
            printf("握手失败: 重试次数过多\n");
            closesocket(sock);
            WSACleanup();
            fclose(fp);
            return 1;
        }
        send_pkt(FLAG_SYN, 0, 0, 0ULL, NULL, 0);
        hs_deadline = now + 300; // 300ms 后重传
        hs_retries++;
    }

    // 等待接收SYN|ACK
    if (wait_readable(sock, 50)) {
        sockaddr_in from;
        int fromlen = sizeof(from);
        int n = recvfrom(sock, (char*)rbuf, (int)sizeof(rbuf), 0, (
            sockaddr*)&from, &fromlen);
        if (n <= 0) continue;

        Header h;
        const uint8_t* payload = NULL;
        uint16_t len = 0;
        if (!parse_packet(rbuf, n, &h, &payload, &len)) continue;

        uint16_t flags = ntohs(h.flags);
        uint32_t ack = ntohl(h.ack);

        if ((flags & (FLAG_SYN | FLAG_ACK)) == (FLAG_SYN | FLAG_ACK))
```



```

        && ack == 1) {
            // 发送最后 ACK
            send_pkt(FLAG_ACK, 0, 1, OULL, NULL, 0);
            connected = 1;
            printf("连接建立成功。\\n");
            break;
        }
    }
}

```

1. 目标与约定

控制位由FLAG_SYN/FLAG_ACK表示；发送端握手的初始序号固定为seq = 0（没有随机ISN），接收端在 SYN|ACK 中通过ack字段告诉下一步期望的第一个数据段号（这里期望为1）。

2. 关键变量与初始化

- hs_deadline：下一次发送（或重传）SYN 的时间点（ms）。
- hs_retries：已经重传的次数，上限为 10。
- connected：连接建立成功的标志。
- 初始动作：hs_deadline = now_ms()（立即发送一次），循环直到connected。

3. 发送与重传逻辑

在循环中，当now >= hs_deadline时会：检查hs_retries >= 10，若满足则超出重试次数，打印失败并退出；发送 SYN (send_pkt(FLAG_SYN, 0, 0, OULL, NULL, 0))；设置下一重传时间hs_deadline = now + 300（固定 300 ms 重传间隔）；增加hs_retries。注意send_pkt内部有should_drop(lossP)，用于丢包模拟，可能“假装发送成功但实际未发出”。

4. 接收 SYN|ACK 与校验

使用非阻塞套接字和wait_readable(sock, 50)轮询接收（50 ms 超时）。收到数据后用parse_packet()校验包格式、magic、版本与校验和。接着取出flags和ack，进行判断。判断条件：(flags & (FLAG_SYN | FLAG_ACK)) == (FLAG_SYN | FLAG_ACK) && ack == 1，即要求包同时带有 SYN 和 ACK 位，且对方的ack等于 1（确认了我们的初始seq = 0并期望下一个seq = 1）。

5. 发送第三次 ACK 并完成连接

满足上述条件后，发送最终 ACK：send_pkt(FLAG_ACK, 0, 1, OULL, NULL, 0) (ack=1)。然后将connected = 1，跳出握手循环并进入后续数据传输流程。

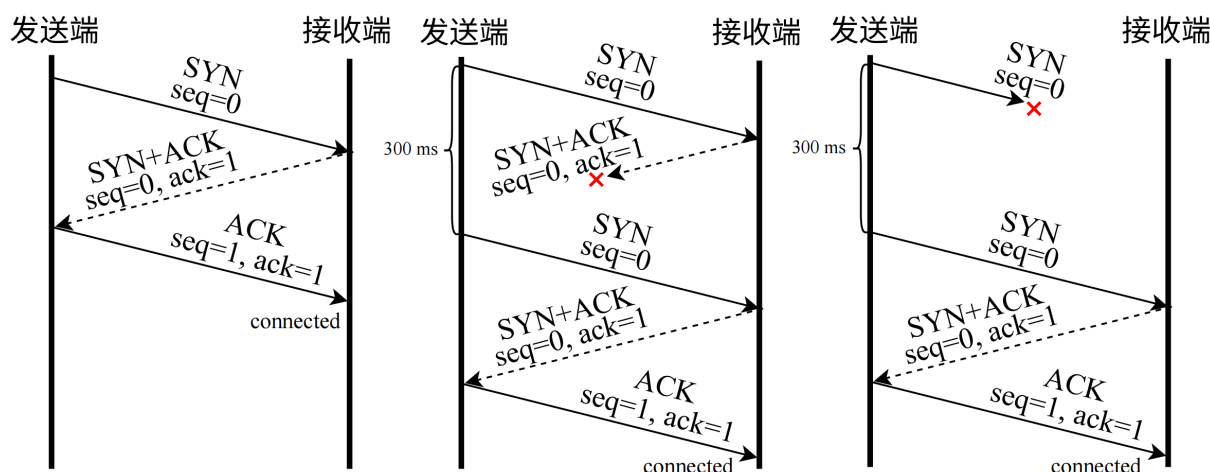


图 4: “三次握手”示意图

3.1.5 在途段管理（流水线核心）

sender.cpp

```
typedef struct InFlightSeg {
    uint32_t seq;           // 段号
    int used;               // 槽位是否被占用
    int acked;              // 段是否已确认
    uint16_t len;           // payload长度
    uint8_t* data;          // payload缓存（用于重传）
    uint64_t sent_time;     // 最近一次发送时间（ms）
    int tx_count;           // 已发送次数
} InFlightSeg;
// ...
const int INFLIGHT_CAP = 128;
InFlightSeg* inflight = (InFlightSeg*)calloc(INFLIGHT_CAP, sizeof(
    InFlightSeg));
```

本次实验要求使用流水线方式，即一个窗口内可以同时发出多段数据，而不是“发一段等一段 ACK”。这会带来两个要求：必须缓存已发送但未确认的数据段，必须能快速定位“某个段是否已确认、是否需要重传、是否还在窗口内”。所以 sender.cpp 用一个“在途段表”来管理当前窗口内发出的所有段。

结构体 InFlightSeg 表示一个“在途段”，用于保存已经发送但尚未被累积 ACK 或 SACK 确认的段，支持重传与统计。inflight 是指向 InFlightSeg 固定大小数组的指针（在代码中用 calloc 分配，容量由 INFLIGHT_CAP 控制，这里固定为 128）。整个发送过程通过该数组管理所有未完全确认的段。

sender.cpp

```
// 在inflight数组中查找某段
static InFlightSeg* inflight_find(InFlightSeg* arr, int cap, uint32_t
seq) {
    for (int i = 0; i < cap; ++i) {
        if (arr[i].used && arr[i].seq == seq) return &arr[i];
    }
    return NULL;
}

// 找一个空槽位
static InFlightSeg* inflight_alloc(InFlightSeg* arr, int cap) {
    for (int i = 0; i < cap; ++i) {
        if (!arr[i].used) return &arr[i];
    }
    return NULL;
}

// 释放一个段槽位 (释放data)
static void inflight_free_slot(InFlightSeg* s) {
    if (!s) return;
    if (s->data) {
        free(s->data);
        s->data = NULL;
    }
    s->used = 0;
    s->acked = 0;
    s->seq = 0;
    s->len = 0;
    s->sent_time = 0;
    s->tx_count = 0;
}
```

这几个函数是辅助函数: `inflight_find()` 函数线性扫描数组, 找到 `used && seq == 目标seq` 的槽位。 `inflight_alloc()` 找到第一个 `used == 0` 的槽位。 `inflight_free_slot()` 释放槽, 并且会 `free(s->data)`, 把 payload 内存释放掉。

3.1.6 滑动窗口与流量控制 (固定 W)

sender.cpp

```
auto send_segment = [&](InFlightSeg* s) -> void {
    if (!s || !s->used || !s->data) return;
    send_pkt(FLAG_DATA, s->seq, 0, 0ULL, s->data, s->len);
    s->sent_time = now_ms();
    s->tx_count++;
    if (!started) { started = 1; t0 = now_ms(); }
};

auto try_send_more = [&]() -> void {
    int flight_limit = (cwnd < W) ? cwnd : W;
    uint32_t wnd_end = base + (uint32_t)flight_limit; // [base,
    wnd_end)
    while (nextSend <= totalSeg && nextSend < wnd_end) {
        // 若已在inflight, 说明之前发过 (一般不会发生), 跳过
        InFlightSeg* ex = inflight_find(inflight, INFLIGHT_CAP,
            nextSend);
        if (ex) {
            nextSend++;
            continue;
        }

        InFlightSeg* slot = inflight_alloc(inflight, INFLIGHT_CAP);
        if (!slot) {
            // 理论上不应发生 (容量足够), 发生则暂停填充
            break;
        }

        memset(slot, 0, sizeof(InFlightSeg));
        slot->used = 1;
        slot->seq = nextSend;
        slot->acked = 0;

        // 从文件加载该段数据到内存 (用于重传)
        if (!load_segment_from_file(fp, fsz, MSS, nextSend, slot)) {
            inflight_free_slot(slot);
            break;
        }

        // 发送该段
        send_segment(slot);
        nextSend++;
    }
}
```

```
};
```

用try_send_more接收了一个匿名函数,首先设置允许的并发在途段数flight_limit为cwnd和W的最小值。发送区间是[base, base + flight_limit);循环从nextSend向上分配槽位、加载数据并调用send_segment(slot)直至超出wnd_end或文件结束或槽位耗尽。保证nextSend >= base (在滑动base后调整)。

send_segment()通过send_pkt()发送,更新s->sent_time = now_ms()、s->tx_count++,并在第一次发送时记录t0。

3.1.7 选择确认 (SACK) 的处理

sender.cpp

```
auto slide_base = [&]() -> int {
    int moved = 0;
    while (base <= totalSeg) {
        InFlightSeg* s = inflight_find(inflight, INFLIGHT_CAP, base);
        if (s && s->used && s->acked) {
            inflight_free_slot(s);
            base++;
            moved++;
        }
        else {
            break;
        }
    }
    // nextSend至少不能落后于base
    if (nextSend < base) nextSend = base;
    return moved;
};
```

// 处理ACK/SACK (关键): ackno为“下一个期望段号”

```
auto process_ack = [&](uint32_t ackno, uint64_t sack_mask) -> void {
    int advanced = (ackno > lastAck);
    int newlyAked = 0;

    // 1) 累积确认: 所有 < ackno 的段都已收到
    if (ackno > 1) {
        uint32_t upto = ackno - 1;
        if (upto > totalSeg) upto = totalSeg;
        // 只需要从base开始标记即可 (更高效)
        for (uint32_t s = base; s <= upto; ++s) {
            newlyAked += mark_acked(s);
        }
    }
};
```

```

}

// 2) SACK位图: bit i=1 表示 (ackno+i) 已收到 (乱序缓冲中)
if (sack_mask != OULL) {
    for (int i = 0; i < 64; ++i) {
        if ((sack_mask >> i) & 1ULL) {
            uint32_t s = ackno + (uint32_t)i;
            if (s >= 1 && s <= totalSeg) {
                newlyAacked += mark_acked(s);
            }
        }
    }
}

// 3) 尝试滑动窗口base (释放已确认段)
slide_base();
// ...
};

```

用process_ack接收匿名函数, ackno表示接收端期望的下一个段号(累积ACK)。接下来进行累积确认,将 [base, ackno-1]的段依次标记为acked(函数mark_acked()调用inflight_find(),找到并设置s->acked = 1)。随后在sack_mask中,对每个 bit=1,将seq = ackno + i标记为acked (只在1至totalSeg范围内)。然后调用slide_base(),只要Base对应槽存在且acked,就inflight_free_slot并base++。同时确保 nextSend >= base。

3.1.8 拥塞控制: Reno 算法

```

sender.cpp

// Reno参数 (单位: 段)
int cwnd = 1;
int ssthresh = 64;
int fastRecovery = 0;
int ca_acc = 0; // 拥塞避免累加器 (实现“每RTT+1”)
// ...
// 拥塞控制: 收到“新ACK (ack前进)”后根据Reno更新cwnd
auto reno_on_new_ack = [&](int newlyAackedCount) -> void {
    if (newlyAackedCount <= 0) return;

    if (cwnd < ssthresh) {
        // 慢启动: 每收到一个新确认, cwnd+1 (近似指数增长)
        cwnd += newlyAackedCount;
    }
    else {

```

```

        // 拥塞避免：每个RTT增长约1，这里用累加器实现
        ca_acc += newlyAkedCount;
        while (ca_acc >= cwnd) {
            ca_acc -= cwnd;
            cwnd += 1;
        }
    }

    if (cwnd < 1) cwnd = 1;
};
// ...
auto process_ack = [&](uint32_t ackno, uint64_t sack_mask) -> void {
    // ...
    // 4) Reno逻辑：区分新ACK与重复ACK
    if (advanced) {
        // 新ACK：可能从快恢复退出
        if (fastRecovery) {
            // Reno：收到新ACK，退出快恢复，cwnd = ssthresh
            cwnd = ssthresh;
            fastRecovery = 0;
            dupAckCnt = 0;
            ca_acc = 0;
        }
        else {
            dupAckCnt = 0;
        }
        lastAck = ackno;
        reno_on_new_ack(newlyAked);
    }
    else {
        // 重复ACK：ack不前进
        dupAckCnt++;

        if (!fastRecovery && dupAckCnt == 3) {
            // 触发快重传/快恢复
            ssthresh = cwnd / 2;
            if (ssthresh < 2) ssthresh = 2;

            // Reno：cwnd = ssthresh + 3
            cwnd = ssthresh + 3;
            fastRecovery = 1;

            // 立即重传“缺失段”：通常就是 ackno 段
            InFlightSeg* miss = inflight_find(inflight, INFLIGHT_CAP,

```

```

        ackno);
        if (miss && miss->used && !miss->acked) {
            send_segment(miss);
            retrans++;
        }
    }
    else if (fastRecovery) {
        // 快恢复期间：每多一个dupACK，cwnd再+1，允许多发一个新段
        cwnd += 1;
    }
}

if (cwnd < 1) cwnd = 1;
};

```

1. 关键变量

- `int cwnd = 1`: 初始拥塞窗口（慢启动从 1 段开始）。
- `int ssthresh = 64`: 初始慢启动阈值。
- `int fastRecovery = 0`: 是否处于快恢复阶段（相当于是布尔标志）。
- `int ca_acc = 0`: 拥塞避免的累加器，实现“每 RTT 增长约 1”。由于代码中 `cwnd`、`ACK` 处理都是整数，无法直接做小数增量。`ca_acc` 把每次收到的新确认数累积起来，只有当累积量达到当前 `cwnd` 时，才把 `cwnd` 增加 1。这样在连续若干次 `ACK` 到来后，总体上每 RTT 增加约 1（因为在一个 RTT 内大约会收到 `cwnd` 个段的确认）。

2. 慢启动与拥塞避免

在收到“新 ACK”时调用 `reno_on_new_ack()`: 若 `cwnd < ssthresh`, 则处于慢启动, `cwnd += newlyAackedCount` (每收到一个新确认, `cwnd` 增加 1), 实现近似指数增长 (`cwnd += newlyAackedCount`)。若 `cwnd` 达到 `ssthresh`, 则进入拥塞避免状态, 使用累加器 `ca_acc += newlyAackedCount`, 当 `ca_acc >= cwnd` 时 `cwnd++` 并 `ca_acc -= cwnd`, 从而实现每 RTT 增长约 1 (见 `reno_on_new_ack` 实现)。

3. 重复 ACK 的快速重传与快速恢复

在 `process_ack()` 中判断为“新 ACK” (`advanced = (ackno > lastAck)`) 还是重复 ACK。若是重复 ACK (`ackno` 没前进), 则 `dupAckCnt++`; 若 `!fastRecovery && dupAckCnt == 3`, 触发三次重复 ACK 快速重传与快速恢复: 计算 `ssthresh = cwnd / 2` (下限调整: 若 `< 2` 则设为 2); 将 `cwnd = ssthresh + 3`, 并设置 `fastRecovery = 1` (代码片段在 `process_ack`); 接着立即重传“缺失段”, 用 `inflight_find(inflight, INFLIGHT_CAP, ackno)` 找到缺失段并 `send_segment(miss)` (实现快重传)。最后快恢复, 每收到额外的重复 ACK, `cwnd += 1` (允许发送更多新的段)。而收到前进的 ACK (新 ACK) 时, 若处于 `fastRecovery`: 退出快恢复, 设置 `cwnd = ssthresh`, `fastRecovery=0`, `dupAckCnt=0`, `ca_acc=0` (即以 `ssthresh` 回退); 否则仅 `dupAckCnt = 0`。最终更新 `lastAck = ackno` 并调用 `reno_on_new_ack()`。

4. 超时处理 (退回到慢启动)

主循环通过查找Base的InFlightSeg并比较 $\text{now} - \text{bs} \rightarrow \text{sent_time} > \text{RTO}$ 来检测超时(仅为单一计时器策略, 检查最老未确认段)。超时时调用`on_timeout()`, 设置 $\text{ssthresh} = \text{cwnd} / 2$ (下限 2), $\text{cwnd} = 1$, 退出快恢复并清除重复ACK计数与累加器。接着重传Base段(`send_segment(s)`), 并将 $\text{nextSend} = \text{base}$ (RTO 后从base重新发送新段)。最后统计 $\text{retrans}++$ 。

对于以上讲的这么多内容, 举一个具体的例子说明。我们假设固定发送窗口 $W = 8$, 设置拥塞窗口 $\text{cwnd} \geq 8$ (不成为限制因素)。初始状态时 $\text{base} = 1$, $\text{nextSend} = 1$ 。假设由于网络异常, 导致段 3 丢失。确认重传机制如图5所示。

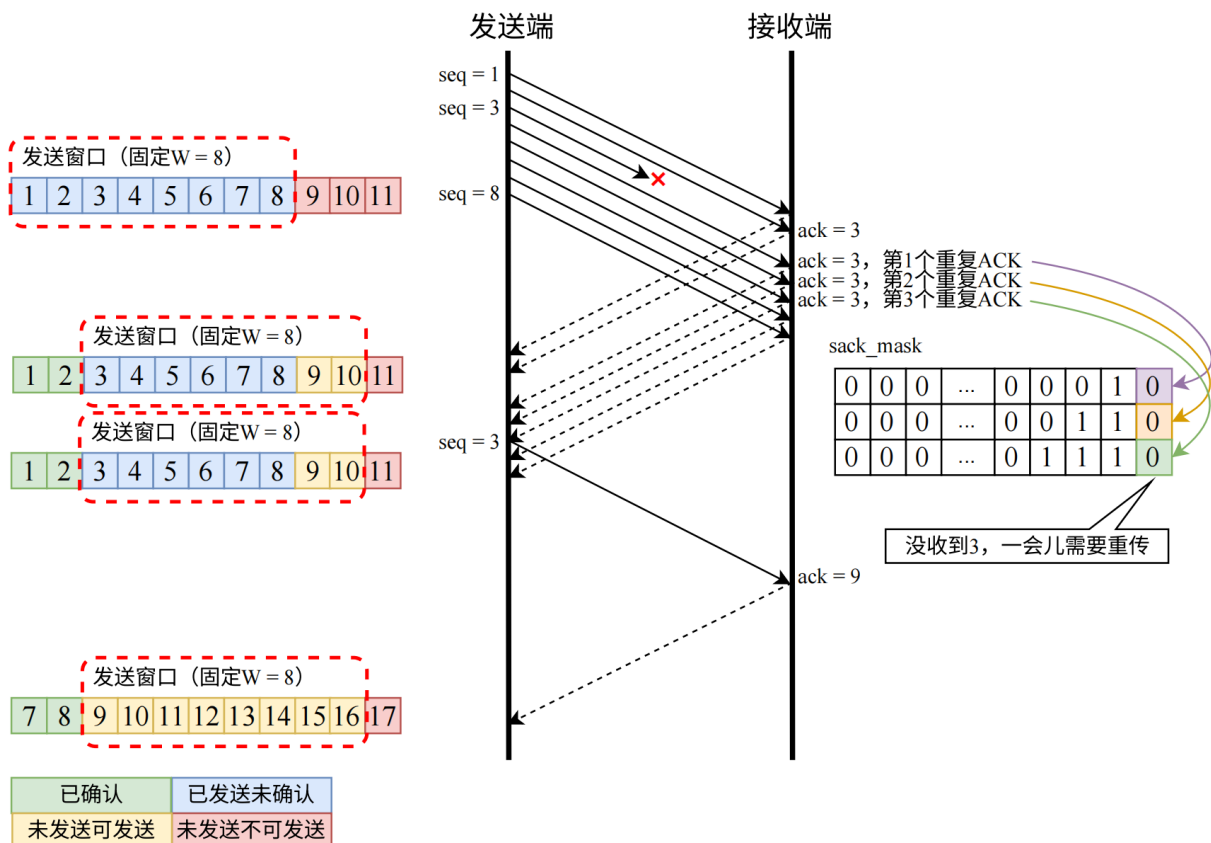


图 5: 确认重传机制示例

3.1.9 连接关闭：四次挥手

sender.cpp

```
int gotFinAck = 0;
int gotPeerFin = 0;
int fin_retries = 0;
uint64_t fin_deadline = now_ms(); // 立即发FIN

while (!(gotFinAck && gotPeerFin)) {
    uint64_t now = now_ms();

    if (now >= fin_deadline) {
```

```

    if (fin_retries >= 10) {
        printf("挥手失败: 重试次数过多\n");
        break;
    }
    // 若还没收到FIN的ACK, 就重发FIN; 若已收到ACK, 就主要等对端FIN
    if (!gotFinAck) {
        send_pkt(FLAG_FIN, finSeq, 0, 0ULL, NULL, 0);
    }
    fin_deadline = now + 300;
    fin_retries++;
}

if (wait_readable(sock, 50)) {
    sockaddr_in from;
    int fromlen = sizeof(from);
    int n = recvfrom(sock, (char*)rbuf, (int)sizeof(rbuf), 0, (
        sockaddr*)&from, &fromlen);
    if (n <= 0) continue;

    Header h;
    const uint8_t* payload = NULL;
    uint16_t len = 0;
    if (!parse_packet(rbuf, n, &h, &payload, &len)) continue;

    uint16_t flags = ntohs(h.flags);

    if ((flags & FLAG_ACK) && !gotFinAck) {
        uint32_t ackno = ntohl(h.ack);
        if (ackno >= finSeq + 1) {
            gotFinAck = 1;
        }
    }

    if (flags & FLAG_FIN) {
        gotPeerFin = 1;
        uint32_t peerFinSeq = ntohl(h.seq);
        // 回ACK确认对端FIN
        send_pkt(FLAG_ACK, 0, peerFinSeq + 1, 0ULL, NULL, 0);
    }
}
}

```

1. 进入关闭阶段的触发

在主发送循环结束后, 发送端进入连接关闭逻辑: 计算 FIN 序号 `uint32_t finSeq = totalSeg + 1,`

并打印printf("开始关闭连接（挥手）...\n"）。

2. 发送 FIN 并启动重传定时器

代码用两个标志追踪进度:gotFinAck = 0和gotPeerFin = 0,并用fin_deadline = now_ms() + fin_retries = 0管理重发。每次到达超时时间(now >= fin_deadline)时,如果还未收到receiver对sender FIN的ACK(!gotFinAck),则调用send_pkt(FLAGS_FIN, finSeq, 0, 0ULL, NULL, 0)发送(或重发)FIN,然后更新fin_deadline = now + 300; fin_retries++;最多重试10次,否则打印失败并退出循环。

3. 等待并处理接收端对 FIN 的 ACK

循环内通过wait_readable(sock, 50)、recvfrom()和parse_packet()读取并验证报文。若报文包含FLAG_ACK且尚未设置gotFinAck,则取出累积确认号ackno,如果ackno >= finSeq + 1则认为对方确认了我方FIN,执行gotFinAck = 1。

4. 接收接收端 FIN 并立即回 ACK

同一接收路径当检测到flags & FLAG_FIN时,设置gotPeerFin = 1,读取接收端FIN的序号peerFinSeq,然后立即回送确认peerFinSeqsend_pkt(FLAGS_ACK, 0, peerFinSeq+1, 0ULL, NULL, 0)(使用接收端FIN序号+1作为ack字段)。这保证对端知道其FIN已被确认。最后当gotFinAck && gotPeerFin为真时,循环退出。“四次挥手”的示意图如图6所示。

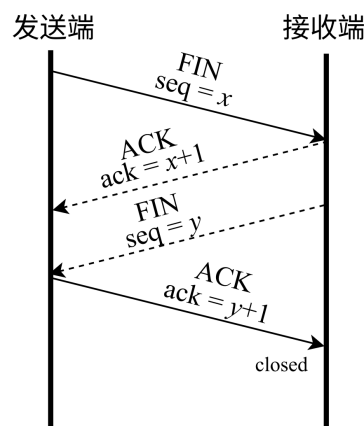


图 6: “四次挥手”示意图

3.2 接收端 receiver.cpp

接收端 receiver.cpp 的实现逻辑很多都和 sender.cpp 一致,不再赘述。以下只介绍 receiver.cpp 特有的关键部分。

3.2.1 接收窗口与乱序缓存

receiver.cpp

```
typedef struct RecvSlot {
    int used;           // 槽位是否被占用
    uint32_t seq;       // 该槽保存的段号
    uint16_t len;       // 数据长度
    uint8_t* data;      // 指向payload的内存
```

```

} RecvSlot;
// ...
int W = atoi(argv[3]);
uint32_t rcv_nxt = 1;
RecvSlot* slots = calloc(SLOT_CAP, sizeof(RecvSlot));

```

接收端用了一个RecvSlot结构体数组来进行乱序缓存。rcv_nxt是当前已经按序交付到应用层的最后一个段号 + 1，表示“期望收到的下一个段号”。

3.2.2 数据接收逻辑

receiver.cpp

```

if (flags & FLAG_DATA) {
    // 固定窗口流控：只接收[rcv_nxt, rcv_nxt+W-1]范围内的段
    if (seq < rcv_nxt) {
        // 旧包（已经交付过），直接回ACK（帮助发送端收敛）
        send_ack();
        continue;
    }
    if (seq >= rcv_nxt + (uint32_t)W) {
        // 超出接收窗口，丢弃并回ACK（让发送端知道窗口边界）
        send_ack();
        continue;
    }

    if (seq == rcv_nxt) {
        // 按序到达：直接写文件
        if (len > 0) {
            fwrite(payload, 1, len, fp);
        }
        rcv_nxt++;

        // 尝试把乱序缓存中连续的段按序交付
        while (1) {
            RecvSlot* s = slot_find(slots, SLOT_CAP, rcv_nxt);
            if (!s) break;
            if (s->len > 0 && s->data) {
                fwrite(s->data, 1, s->len, fp);
            }
            slot_free(s);
            rcv_nxt++;
        }
    }
    else {

```

```

// 乱序到达：缓存（若未缓存过）
if (!slot_find(slots, SLOT_CAP, seq)) {
    RecvSlot* slot = slot_alloc(slots, SLOT_CAP);
    if (slot) {
        uint8_t* buf2 = NULL;
        if (len > 0) {
            buf2 = (uint8_t*)malloc(len);
            if (buf2) memcpy(buf2, payload, len);
        }
        slot->used = 1;
        slot->seq = seq;
        slot->len = len;
        slot->data = buf2;
    }
    // 如果缓存满了，直接丢弃（仍然回ACK+SACK，让发送端决定重传）
}

// 回ACK + SACK位图（选择确认）
send_ack();
continue;
}

```

接收端只接受序号落在 $[rcv_nxt, rcv_nxt + W - 1]$ 的 DATA 段：若 $seq < rcv_nxt$ （已交付的旧包），直接回 ACK（帮助发送端收敛）；若 $seq \geq rcv_nxt + W$ （超出窗口），丢弃并回 ACK（告知窗口边界）。按序到达的时候（ $seq == rcv_nxt$ ），直接写入文件（`fwrite`）， rcv_nxt++ ，然后循环检查乱序缓存（`slots`）是否有下一个连续段并交付，直到无法继续。乱序到达的时候（ $rcv_nxt < seq < rcv_nxt + W$ ），缓存到一个槽位（`RecvSlot`），避免重复缓存（先`slot_find`），使用`slot_alloc`找空槽，`payload`使用`malloc`存储；若槽满则直接丢弃（仍会发送 ACK+SACK 让发送端重传）。

3.2.3 SACK 位图的生成

```

receiver.cpp

// 计算SACK位图：bit i=1 表示 (rcv_nxt+i) 已缓存（乱序到达）
auto build_sack_mask = [&]() -> uint64_t {
    uint64_t mask = 0ULL;
    int limit = W;
    if (limit > 64) limit = 64;
    for (int i = 0; i < limit; ++i) {
        uint32_t s = rcv_nxt + (uint32_t)i;
        if (slot_find(slots, SLOT_CAP, s)) {

```

```

        mask |= (1ULL << i);
    }
}
return mask;
};

```

首先将位图初始化为 0，表示暂时认为“没有任何乱序段被接收”。接着限制扫描范围为 $\text{limit} \leq 64$ ，因为位图只有 64 位，即使 $W > 64$ ，SACK 也最多表达 ack 之后 64 个段的状态。然后从`rcv_nxt`开始扫描乱序缓存：调用`slot_find()`函数，如果返回了非 0，那么说明在接收缓存里找到了，也就是这个段已经到达接收端，但没法交付（前面缺段），于是把第 i 位设为 1。注意 0 和 1 要加后缀ULL（Unsigned Long Long，无符号 64 位整数）。因为`mask`的类型是 64 位，0 和 1 默认是`int`（32 位），左移的位数大于等于 32 位时会出现未定义行为。

4 运行演示

4.1 编译

我使用的操作系统是 **Windows 11 家庭中文版**，编译器是 **Visual Studio 2026**。环境与配置方法和实验 1 完全一致。编译生成 `sender.exe` 与 `receiver.exe`，如图7所示。

名称	修改日期	类型	大小
 receiver.exe	2025/12/20 22:40	应用程序	15 KB
 receiver.pdb	2025/12/20 22:40	Program Debug Da...	684 KB
 sender.exe	2025/12/20 22:40	应用程序	20 KB
 sender.pdb	2025/12/20 22:40	Program Debug Da...	708 KB

图 7: 编译生成的 exe 文件

程序的启动方法是使用命令行，格式如下：

```
.\receiver.exe <listen_port> <output_file> <W> [--loss=0.0] [--delay=0.0]
```

```

.\sender.exe <receiver_ip> <port> <input_file> <W> [--mss=1000] [--rto=300]
    [--loss=0.0] [--delay=0.0]

```

4.2 运行（无丢包）

我使用 Windows 11 主机作为发送端，Windows 10 虚拟机作为接收端，主机和虚拟机处于同一局域网下。分别执行`ipconfig`，得到主机（发送端）的 IP 地址为 192.168.188.128，虚拟机（接收端）的 IP 地址为 192.168.188.144。

```

以太网适配器 VMware Network Adapter VMnet8:

连接特定的 DNS 后缀 . . . . . : 
本地链接 IPv6 地址. . . . . : fe80::67b4:a603:6795:6a79%10
IPv4 地址 . . . . . : 192.168.188.128
子网掩码 . . . . . : 255.255.255.0
默认网关. . . . . : 192.168.188.2

```

图 8: 主机（发送端）的 IP 地址

```

以太网适配器 Ethernet0:

连接特定的 DNS 后缀 . . . . . : localdomain
本地链接 IPv6 地址. . . . . : fe80::5420:1d5b:ad66:803b%9
IPv4 地址 . . . . . : 192.168.188.144
子网掩码 . . . . . : 255.255.255.0
默认网关. . . . . : 192.168.188.2

```

图 9: 虚拟机（接收端）的 IP 地址

将助教发的测试文件 1.jpg、2.jpg、3.jpg 和 helloworld.txt 放在和 sender.exe 同一目录下。以窗口大小 W = 32 为例。

我使用 9000 端口，先启动接收端：执行命令 `.\receiver.exe 9000 1.jpg 32`；再启动发送端，执行命令 `.\sender.exe 192.168.188.144 9000 1.jpg 32`。可以看到图片 1.jpg 被完整地传到了虚拟机，如图10所示。还可以主机和虚拟机分别执行 `certutil -hashfile 1.jpg MD5` 计算 MD5 哈希值，发现主机和虚拟机上的 1.jpg 的哈希值一模一样，如图11所示。说明我设计的程序能够正确传输文件，并且实现了可靠传输。

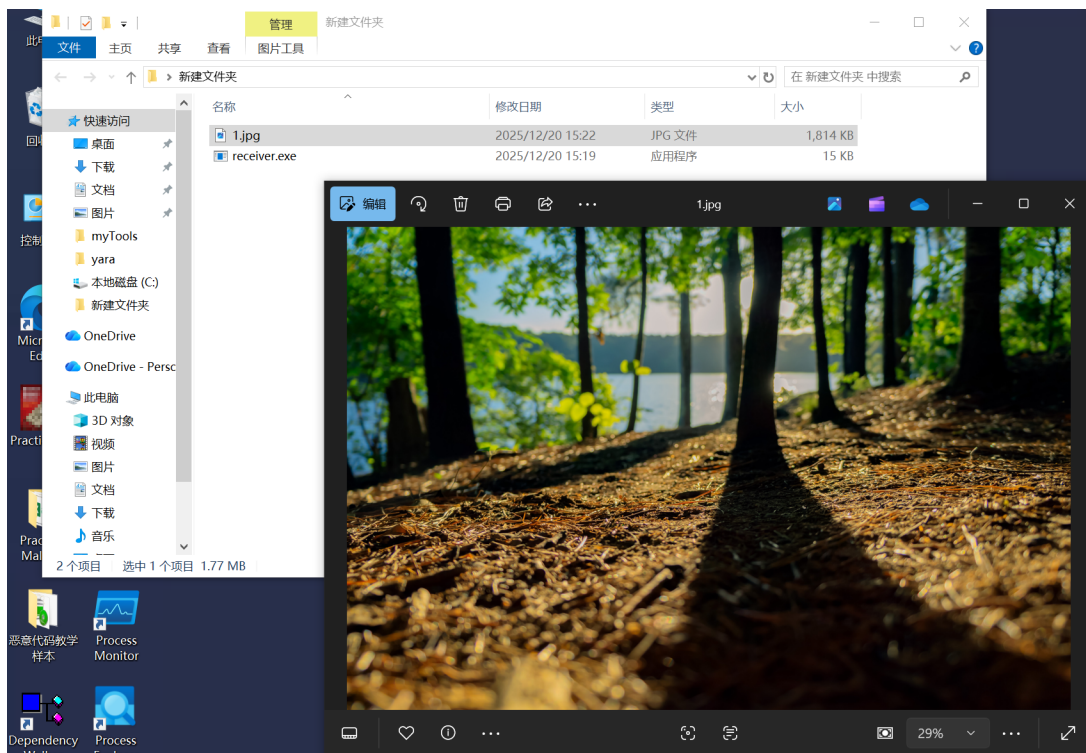


图 10: 1.jpg 被成功传输


```
重传次数: 0
D:\南开批事情\第5学期-计算机网络\实验\实验2 设计可靠传输协议并编程实现>certutil -hashfile 1.jpg MD5
MD5 的 1.jpg 哈希:
2f763f23ddf5e481ba2c8755b56e097e
CertUtil: -hashfile 命令成功完成。
D:\南开批事情\第5学期-计算机网络\实验\实验2 设计可靠传输协议并编程实现>

此电脑 收到FIN, 开始关闭连接...
3D 对象 连接关闭完成。
视频 C:\Users\36112\Desktop\新建文件夹>certutil -hashfile 1.jpg MD5
图片 MD5 的 1.jpg 哈希:
文档 2f763f23ddf5e481ba2c8755b56e097e
下载 CertUtil: -hashfile 命令成功完成。
音乐 C:\Users\36112\Desktop\新建文件夹>
```

图 11: 1.jpg 的哈希值

程序输出的日志如图12和图13所示。

```
D:\南开批事情\第5学期-计算机网络\实验\实验2 设计可靠传输协议并编程实现>.\sender.exe 192.168.188.144 9000 1.jpg 32
文件大小=1857353 bytes, 总段数=1858, MSS=1000, 固定窗口W=32, RT0=300ms, loss=0.000
开始连接 (3次握手) ...
连接建立成功。
开始发送数据...
所有数据段均已确认。
开始关闭连接 (挥手) ...
传输完成。
传输时间: 234 ms
平均吞吐率: 63.499 Mbps
重传次数: 0
```

图 12: sender.exe 的输出日志

```
C:\Users\36112\Desktop\新建文件夹>.\receiver.exe 9000 1.jpg 32
接收端监听端口: 9000
等待连接 (握手) ...
连接建立成功。
收到FIN, 开始关闭连接...
连接关闭完成。
```

图 13: receiver.exe 的输出日志

我接着测试了 2.jpg、3.jpg 和 helloworld.txt 这几个文件，发现都可以正确传输。

4.3 探索不同发送窗口和接收窗口大小以及不同丢包率对传输性能的影响

4.3.1 不同发送窗口对传输性能的影响

在探究不同发送窗口对传输性能的影响时，固定接收窗口大小为 32，丢包率设为 0.03，使用图片 3.jpg 进行传输。接收端和发送端分别执行 `.\receiver.exe 9000 3.jpg 32 --loss=0.03` 和 `.\sender.exe 192.168.188.144 9000 3.jpg 32 --loss=0.03`（分别测试 8, 16, 24, 32, 40, 48, 56, 64），最终的结果如表1所示。

表 1: 不同发送窗口对传输性能的影响

发送窗口大小	8	16	24	32	40	48	56	64
传输时间 (ms)	26375	22360	23105	19297	30688	27562	22844	22656
平均吞吐率 (Mbps)	3.630	4.282	4.160	4.962	3.120	3.474	4.192	4.226

当发送窗口较小时,流水线深度不足,链路带宽利用率低,吞吐率较小;随着发送窗口增大,流水线逐渐填满,吞吐率明显提升;当发送窗口进一步增大时,在途数据量增加导致丢包事件频繁发生,Reno 的拥塞控制机制频繁回退,窗口振荡加剧,吞吐率趋于饱和甚至下降。

4.3.2 不同接收窗口对传输性能的影响

在探究不同接收窗口对传输性能的影响时,固定发送窗口大小为 32,丢包率设为 0.03,使用 3.jpg 进行传输。接收端和发送端分别执行 `.\receiver.exe 9000 3.jpg 32 --loss=0.03`(分别测试 8, 16, 24, 32, 40, 48, 56, 64)和 `.\sender.exe 192.168.188.144 9000 3.jpg 32 --loss=0.03`,最终的结果如表2所示。

表 2: 不同接收窗口对传输性能的影响

接收窗口大小	8	16	24	32	40	48	56	64
传输时间 (ms)	59594	23688	29609	27140	25500	18985	24359	17203
平均吞吐率 (Mbps)	1.607	4.402	3.234	3.528	3.755	5.044	3.931	5.566

当接收窗口较小时,发送端受到接收能力限制,流水线深度不足,吞吐率较低;随着接收窗口增大,乱序缓存和选择确认机制能够充分发挥作用,吞吐率明显提升;当接收窗口达到一定规模后,不再成为系统瓶颈,吞吐率趋于稳定。

4.3.3 不同丢包率对传输性能的影响

在探究不同丢包率对传输性能的影响时,固定发送和接收窗口大小为 32,使用 helloworld.txt 进行传输。接收端和发送端分别执行 `.\receiver.exe 9000 helloworld.txt 32 --loss=0.03`和 `.\sender.exe 192.168.188.144 9000 helloworld.txt 32 --loss=0.03`(分别测试 0, 0.01, 0.02, 0.03, 0.06, 0.1, 0.2),最终的结果如表3所示。

表 3: 不同丢包率对传输性能的影响

丢包率	0	0.01	0.02	0.03	0.06	0.1	0.2
传输时间 (ms)	172	468	1797	3500	11187	31094	122782
平均吞吐率 (Mbps)	77.014	28.304	7.371	3.785	1.184	0.426	0.108

随着丢包率的增大,重传次数显著增加,TCP Reno 拥塞控制机制频繁回退,导致有效在途数据量下降,平均吞吐率持续降低,且在高丢包率条件下下降趋势尤为明显。

5 遇到的一个有意思的与实验无关的问题

最开始我在 Windows 11 主机的 Visual Studio 上用 Debug 模式编译，将 Debug 模式生成的 receiver.exe 复制到虚拟机里运行，结果显示如图15所示的报错。

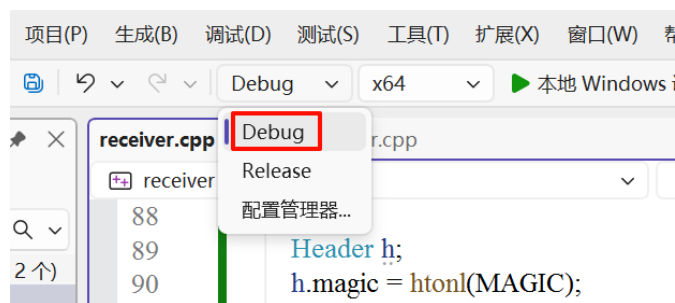


图 14: 使用 Debug 模式编译



图 15: 运行 exe 报错

ucrtbased.dll 是 C/C++ 程序的一个最底层的运行库，仅用于调试（Debug 版），而 ucrtbase.dll 才用于正式发布程序。Windows 默认只带 ucrtbase.dll，不带 ucrtbased.dll。我的主机里安装了 Visual Studio，所以有 ucrtbased.dll 等一整套 Debug C Runtime；而 Win10 虚拟机是“干净系统”，没有 Visual Studio，也就没有 ucrtbased.dll。所以如果想要发布、提交、给别人运行 exe 文件，一定用 Release 编译。